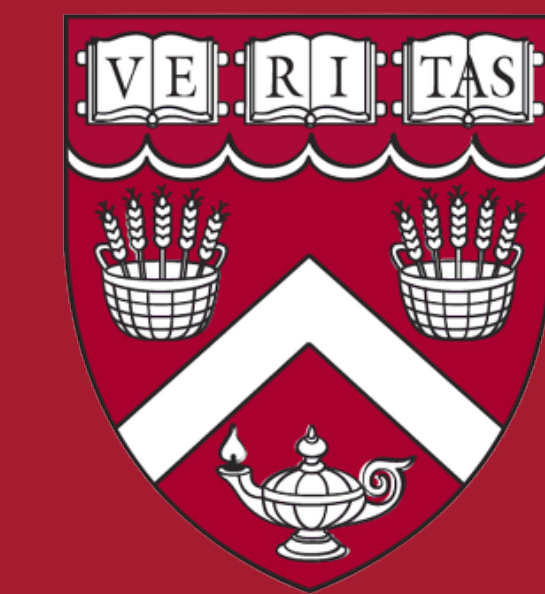




Trajectory-Aware Prompt Optimization for AI Agents

Michael Culver Jinyu Han Jerry Kim Matthew Michel Kaylee Vo

Harvard Extension School - Harvard University



Introduction

Prompt optimization offers a **gradient-free** alternative to reinforcement learning (RL) for improving LLM agents. Instead of updating model weights, our approach optimizes the instructions an agent conditions on, leaving model weights frozen. We use **GEPA**, a reflective prompt optimizer, applied to **DSPy** agent modules operating inside **GEM**, a standardized gym environment with per-turn dense rewards and published RL baselines [1].

Research question: How much of RL’s performance can token-space prompt optimization recover, and at what infrastructure cost?

Motivation

Training agentic LLMs with RL requires gradient updates, rollout infrastructure, and substantial compute, which is costly. Developing **less costly** approaches to agent optimization allows smaller operations access to agent-based technologies.

We evaluate our approach against two reference points: (1) the model’s own **un-optimized prompt**, and (2) **published GEM RL results**.

Contributions

1. A **GEM–DSPy adapter** bridging RL-designed gym environments with token-space prompt optimization.
2. A **trajectory-aware composite fitness function** combining discounted return, loop-detection penalty, and step/call-efficiency bonuses.
3. A full **baseline-vs-optimized benchmark** across four agentic environments, six model families, and three seed replications.

Method

Agents are LLMs embedded in the DSPy module, following the ReAct pattern [2]. GEPA mutates only the instruction text. **Environments:** HotpotQA (multi-hop QA, with/without tool) and Orz57K (math reasoning, with/without tool).

Models:

- Qwen3-1.7B-Base
- Qwen3-4B-Base
- Llama-3.1-8B-Instruct
- Gemma-3-4B-IT
- Mistral-7B-Instruct
- Nemotron-Nano-9B

Training Loop: Iterative Prompt Refinement

1. **Mutate.** A reflection language model rewrites the current system prompt. The solver is the task-model-driven agent, wrapped as a DSPy module. Behavior is entirely parameterized by the prompt GEPA mutates.
2. **Rollout.** For each training example:
 - The solver extracts the current instructions and uses them as its system prompt
 - A fresh episode is initialized in the environment (GEM), producing an initial observation
 - The solver interacts with GEM step by step: it observes the state, selects an action, the environment updates and returns a new observation, reward, and completion signal, and each step is logged
 - The episode ends if a terminal state is reached
 - The result is a complete trajectory log of the episode
3. **Score.** The fitness metric evaluates the trajectory. The composite score is normalized and accompanied by a human-readable breakdown explaining *why* the trajectory received that score.
4. **Update.** Rather than keeping only the single best prompt, the optimizer maintains a *Pareto frontier* of promising prompt candidates (total of 12 candidates). The reflection model reads the score breakdown as feedback to propose the next round of prompt mutations.

Fitness Function

The fitness $F(\tau)$ is implemented as a weighted composite of four terms,

$$F(\tau) = \tilde{F}_{\text{ret}}(\tau) + w_{\text{loop}} F_{\text{loop}}(\tau) + w_{\text{step}} F_{\text{step}}(\tau) + w_{\text{call}} F_{\text{call}}(\tau). \quad (1)$$

1. Normalized Discounted Return:

$$\tilde{F}_{\text{ret}}(\tau) = \frac{R_T \sum_{t=0}^{T-1} \gamma^{T-1-t} + \lambda \sum_{t=0}^{T-1} r_{t+1}}{\sum_{t=0}^{T-1} \gamma^{T-1-t} + \lambda T}, \quad R_T = \mathbb{1}[r_T > 0]. \quad (2)$$

- Discount runs in *reverse time*: the exponent $T-1-t$ assigns full weight $\gamma^0 = 1$ to the final step and discounts earlier steps.
- Gated by R_T , credit from the discounted-return component ($\sum \gamma^{T-1-t}$) accrues only when the trajectory terminates with positive final reward, while the auxiliary per-step term ($\lambda \sum r_{t+1}$) is added unconditionally.

2. Loop-detection Penalty:

$$F_{\text{loop}}(\tau) = -\frac{1}{T-1} \sum_{t=1}^{T-1} \mathbb{1}[a_t \in \{a_s : \max(0, t-W) \leq s < t\}]. \quad (3)$$

- Returns a value in $[-1, 0]$ and discourages identical-action repetition within a sliding window. Window size W is a hyperparameter, set to 3.

3. Step-efficiency Bonus:

$$F_{\text{step}}(\tau) = R_T \cdot \max\left(0, 1 - \frac{T}{T_{\text{max}}}\right), \quad T_{\text{max}} = 50. \quad (4)$$

- For trajectories with $T < T_{\text{max}}$, the agent receives a bonus reward.

4. Call-efficiency Bonus:

$$F_{\text{call}}(\tau) = R_T \cdot \max\left(0, 1 - \frac{\sum_{t=0}^{T-1} (|c_t^{\text{LLM}}| + |c_t^{\text{tool}}|)}{T_{\text{max}} \cdot B}\right), \quad B = 8. \quad (5)$$

- Rewards successful trajectories whose total per-step LLM and tool-call counts $|c_t^{\text{LLM}}| + |c_t^{\text{tool}}|$ remain well within the budget $T_{\text{max}} \cdot B$.

Example of Prompt Evolution

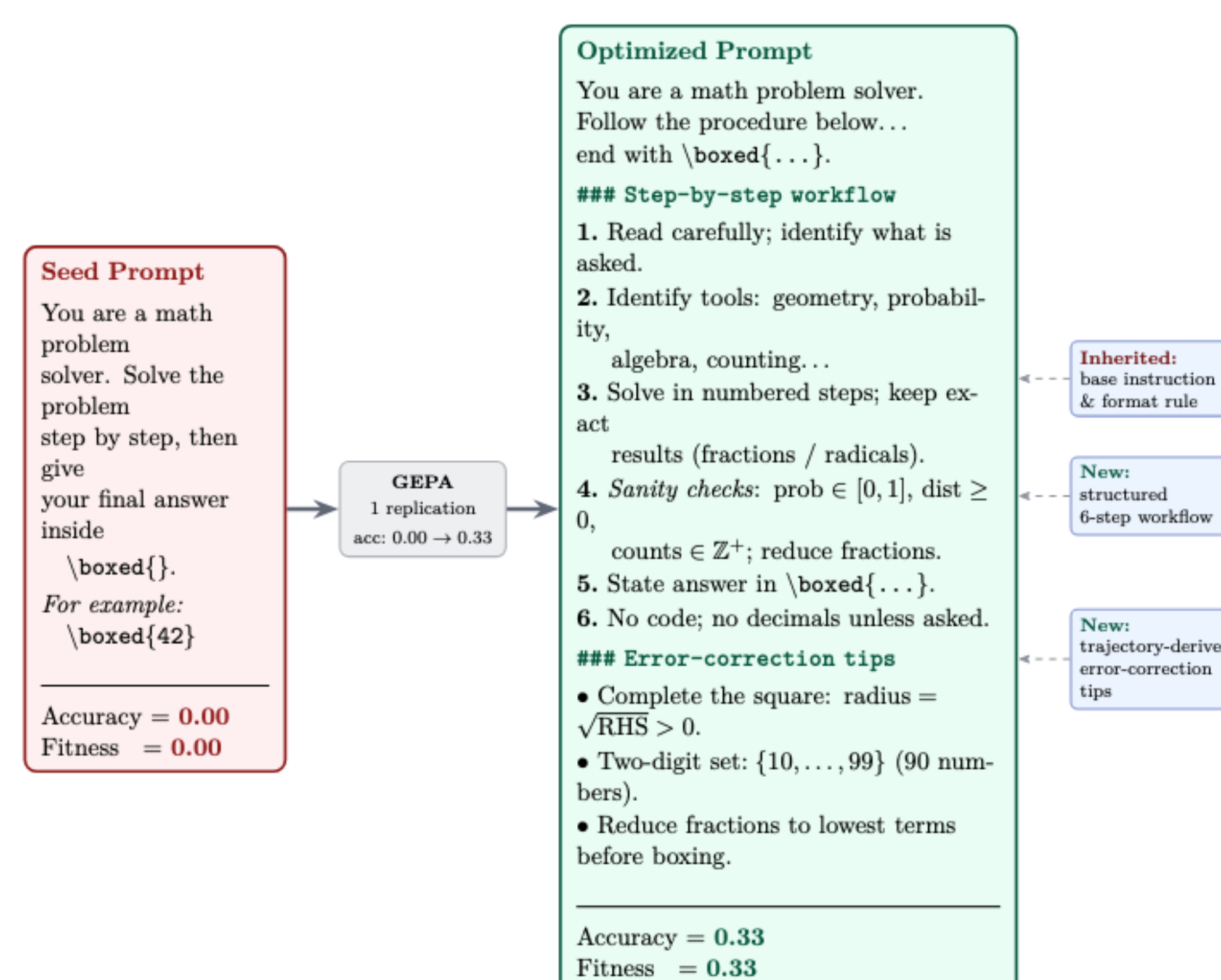


Figure 1. Example of prompt evolution in one GEPA replication on a mathematics benchmark.

Experiment Results

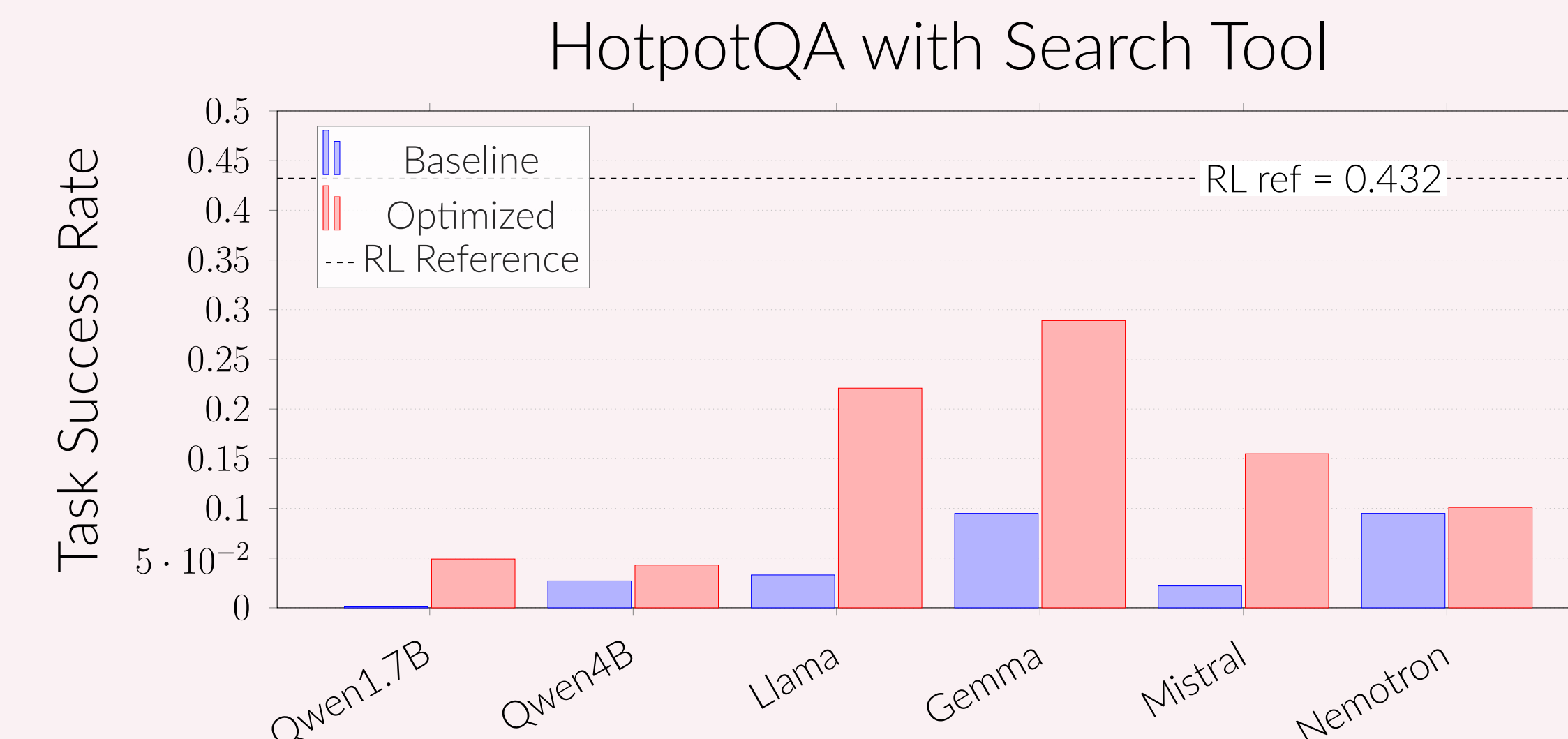


Figure 2. HotpotQA with tool: mean baseline and optimized values vs. RL reference.

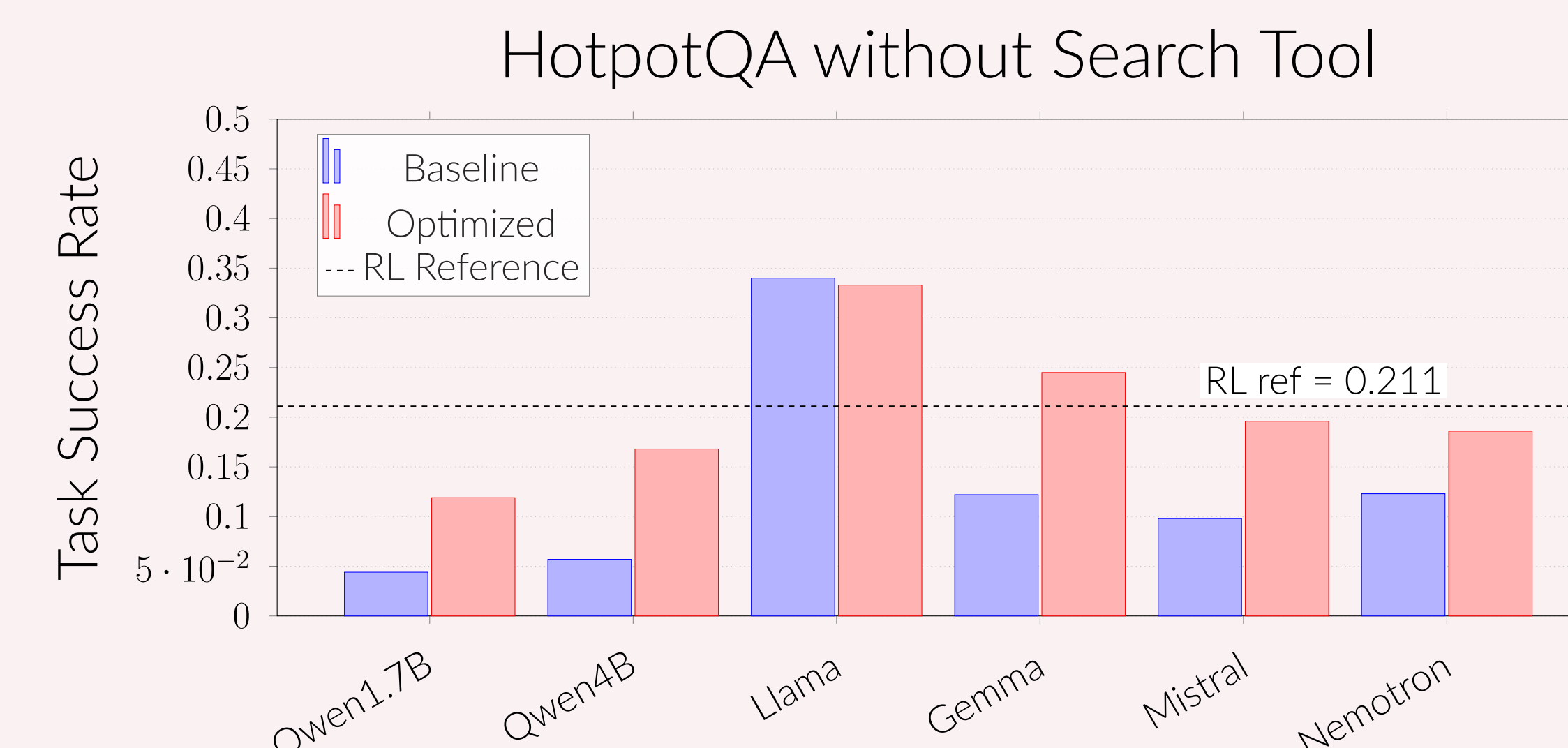


Figure 3. HotpotQA without tool: mean baseline and optimized values vs. RL reference.

The results of our approach **do not match the performance of RL-based methods** given in the GEM paper [1]. We observe improvements over the baseline of simply using unoptimized prompts. Only Gemma without tooling provides a performance boost equivalent to the RL-based methods.

Resource Use & Cost

Mean per-run cost across completed cells: **\$0.35–\$4.45**. No-tool settings are consistently cheaper than with tooling. All runs are cheaper than GPU-based RL training. For HotpotQA-notool, Gemma’s run gained 12.3 percentage points (pp) of accuracy at a cost of \$0.346, giving an efficiency of 35.5 pp/\$.

Environment	Model	Base	Opt.	Cost	Efficiency (pp/\$)
hotpotqa-notool	Gemma-3-4B-IT	0.122	0.245	\$0.346	35.5
hotpotqa-notool	Llama-3.1-8B-Instruct	0.340	0.333	\$1.100	−0.6
hotpotqa-tool	Gemma-3-4B-IT	0.095	0.289	\$0.737	26.3
hotpotqa-tool	Llama-3.1-8B-Instruct	0.033	0.221	\$4.451	4.2

Future Work

1. Isolate the contribution of specific fitness components through ablation studies to distinguish whether the gains arise primarily from dense reward use, loop penalties, efficiency terms, or the reflective mutation process itself.
2. Investigate the boundary between policy and capability shaping more directly.

References

- [1] Zichen Liu, Anya Sims, Keyu Duan, Changyu Chen, Simon Yu, Xiangxin Zhou, Haotian Xu, Shaopan Xiong, Bo Liu, Chenmian Tan, Chuen Yang Beh, Weixun Wang, Hao Zhu, Weiyan Shi, Diyi Yang, Michael Shieh, Yee Whye Teh, Wee Sun Lee, and Min Lin. GEM: A Gym for Agentic LLMs, 2025.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.